

Prism — LLM Gateway and Semantic Cache

Approach one-pager · Asma Sobia · 17 September 2026

The two decisions. Prism puts one OpenAI-compatible API in front of multiple providers and makes two decisions on every request — *how hard is this prompt* (which model tier deserves it) and *have we answered this before* (semantic cache) — while enforcing per-tenant virtual keys, rate limits and cost budgets, metering usage, and streaming without buffering.

REQUEST PATH

key → active → model exists → allowlist → rate limit → budget → route (auto) → cache lookup → dispatch (retry ▶ failover) → meter → log

Checks run cheapest-and-most-certain first, so a refused request costs no embedding and no upstream call. Two orderings are deliberate. **Unknown model is checked before the allowlist** — a 403 for a model that does not exist would confirm its existence to any caller holding any valid key. **Budget exhaustion returns 402, not 429** — the provided load test classifies every 429 as "rate limited", so 429 here would corrupt the one measurement that test exists to produce, and 402 says something true that 429 does not: retrying later cannot help.

BUILD PATH — FIVE SLICES, IN DEPENDENCY ORDER

1. **Spine.** Config loading and validation, schema, key authentication, the error contract. Contract before mechanism: the `x-prism-*` headers and error shapes are fixed first, so nothing downstream invents its own.
2. **Data plane.** Provider adapter behind an interface, model and alias resolution, rate limiter, cost metering, atomic budget charge, request log — including rejections.
3. **Resilience and streaming.** Per-attempt timeouts, retry with full-jitter backoff, failover down the resolved chain; SSE relayed chunk-by-chunk as the raw JSON the provider sent.
4. **Semantic intelligence.** One embedding model used twice — the cache (exact hash → cosine → guards) and the auto router (kNN over hand-authored exemplars) — plus an offline routing eval.
5. **Console and verification.** Ops console, smoke test, load test, routing eval, live provider failure drills, verification report.

DESIGN CHOICES AND THE REASONING BEHIND THEM

- **Postgres, not SQLite.** Budget accounting needs `SET spent = spent + x` to be genuinely atomic. SQLite serialises writers, so a read-then-write implementation would reconcile perfectly *for the wrong reason* — the store is chosen so the concurrency claim is actually put under test.
- **Four tables for seven entities.** Only mutable state that must survive a restart goes in the database: tenants, request log, cache entries, budget counters. Providers, aliases and prices have no per-request mutable state, so they live in config files validated once at startup — no migration to change a price, no join to read one.
- **Usage is a derived query, not a stored aggregate.** A usage row written beside every log row is a dual write, and dual writes drift: the moment one succeeds and the other fails, the usage API and the log disagree with no way to say which is right. One source of truth, aggregated on read.
- **Money is Decimal / NUMERIC(18,10), never float.** Binary floats do not sum associatively, so the same set of costs totals differently depending on the order requests happened to finish. The scale is set by the smallest seeded budget — anything coarser rounds it to zero.
- **Virtual keys are stored as SHA-256 plus a non-secret prefix.** Lookup hashes the presented key and finds the row by index, so a database dump yields no working credential. SHA-256 rather than bcrypt because a slow hash defends *low-entropy, human-chosen* secrets; a virtual key is high-entropy and machine-generated, so a slow hash would buy nothing and cost ~100 ms per request.
- **Cache scope is part of the key, not a filter on it** — (tenant, tier, conversation prefix, sampling parameters). A cross-tenant hit is *not expressible*, so isolation lives in the entry's identity rather than in a `WHERE` clause someone can forget.
- **Cache lookup is a pipeline of rejections.** Exact hash first, so a repeated prompt costs one index lookup and no embedding; then cosine against a per-tenant threshold; then two guards — differing numbers, IDs or amounts, and differing negation. Every stage can only reject, so no guard can *cause* a hit. Time-sensitive prompts are answered and never stored.
- **The embedding model is a vendored artifact.** It must be on disk before startup and is never fetched on the request path — a gateway that reaches a model host mid-request has an unadvertised dependency and an unbounded tail latency.
- **One embedding model serves both features.** The cache and the auto router need the same capability, so they share one warmed instance. Routing is graded offline against a held-out set, with `expected_tier` reaching the scorer and never the classifier.
- **Rate-limit state is in memory and race-safe in-process.** The data model permits this explicitly; what it requires is a counter that cannot over-admit under contention, which a test pins directly. Single-process by design — shared state (Redis) is the multi-process answer and is out of scope.
- **Admission asks whether *any* budget remains, not whether there is enough.** A request's cost is unknown until the provider reports usage, and for a stream until it ends, so there is no figure to reserve against. The resulting overshoot is bounded by one window's requests, and the charge itself is atomic.
- **Failover happens before the first token, never after.** What is worth retrying is decided from the response, not by the retry loop: transient failures retry and fail over, a 400 does neither, an auth or route error fails over without retrying. Once bytes have reached the client the partial answer cannot be un-sent, so a dying stream is terminated with an error event and `[DONE]` rather than spliced onto another provider.

HOW IT GETS PROVEN

Smoke test green on the header, streaming and cache contract. Load test showing **zero over-admission** and accounting that reconciles exactly against the usage API. Routing eval **above the 80% bar and clearly beating a length-only baseline**, with a logged reason per case. Four live failure drills — kill, slow, flaky, restore — with `x-prism-fallback` set and no client left hanging. A coarse added-latency measurement, method documented. Automated tests cover the four flows named critical: cost math, limiter, cache decision, allowlist.

Basis: a prototype spike covering slices 1–5 already exists on the working branch (357 automated tests; routing eval 19/20 = 95% semantic against 8/20 = 40% for a length-only baseline), so the choices above are measured rather than assumed. Verification, the ops console and the report are the remaining slice.